



Chapter 3

Cascading Style Sheet (CSS)

College of Information and Communication Technology
Can Tho University
Dr. Trieu Thanh Ngoan

Content

1. Introduction to CSS
2. CSS levels and syntax
3. CSS selectors
4. CSS properties
5. Inheritance
6. The layout
7. The Box model
8. CSS convention



Content

1. Introduction to CSS
2. CSS levels and syntax
3. CSS selectors
4. CSS properties
5. Inheritance
6. The layout
7. The Box model
8. CSS convention

◆ Why CSS?

HTML markup: used to represent

- **Semantic**: e.g. `<h1>` means top-level heading
- **Presentation**: `<h1>` elements look a certain way (*bold and big*)

It is advisable to separate semantics from presentation:

- It's easier to present documents on multiple platforms (browser, cellphone, ...)
- It's easier to generate documents with consistent look
- Semantic and presentation changes can be made independently of one another

**HTML was NEVER intended to contain tags
for formatting a web page!**

◆ What is CSS?

A language that **describes the style** of an HTML document
describes how HTML elements should be displayed

Cascading Style Sheets **removed the style** formatting from the HTML page

CSS benefits:

- Easier to **maintain** and update
- Greater **consistency** in design
- More **formatting options**
- **Lightweight** code
- **Faster** download times
- Ease of presenting different styles to **different viewers**, etc.



Cons: the
compatibility with
the browsers

What is CSS?

Introduction to CSS

Web page = HTML (structure) + CSS (presentation)

```
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
    <h1>My First CSS Example</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

My First CSS Example

This is a paragraph.

No CSS

```
<style>
  body {
    background-color: lightblue;
  }
  h1 {
    color: white;
    text-align: center;
  }
  p {
    font-family: verdana;
    font-size: 20px;
  }
</style>
```

My First CSS Example

This is a paragraph.

With CSS

Content

1. Introduction to CSS
- 2. CSS levels and syntax**
3. CSS selectors
4. CSS properties
5. Inheritance
6. The layout
7. The Box model
8. CSS convention

◆ Levels of CSS

1. Inline:

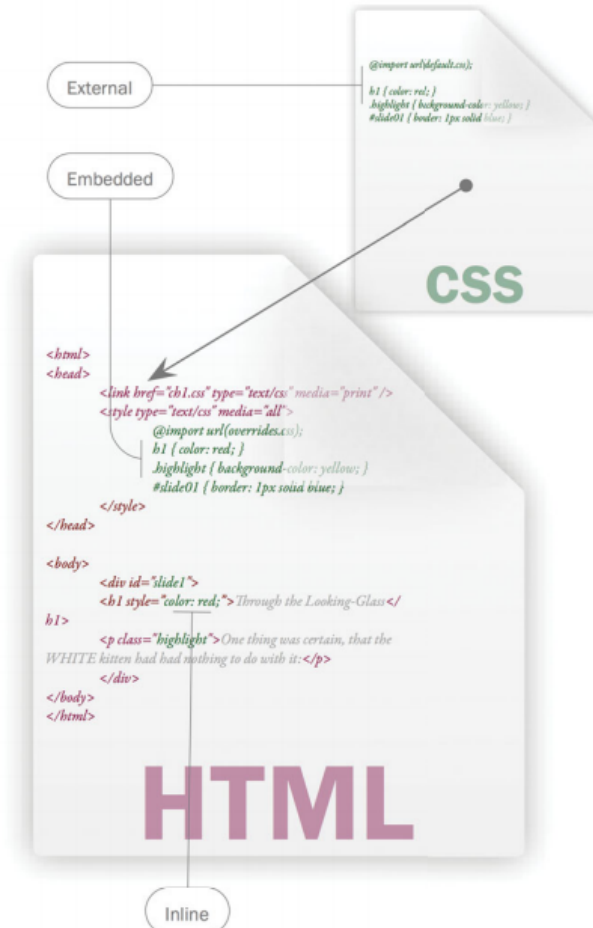
- specified for a **specific occurrence of a tag** and apply only to that tag
- appear **in the tag** itself

2. Internal (document):

- apply to the **whole document** in which they appear
- appear in the **head of the document**

3. External:

- can be applied to **any number of documents** (*entire website*)
- defined in **separate files**



◆ Levels of CSS

1. **Inline:** uses the **style attribute** of the HTML element

General form:

`style="property1: value1; property_2: value2;..."`

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1 style="color:blue;">This is a Blue Heading</h1>
  </body>
</html>
```

◆ Levels of CSS

2. Internal: defined in the **<head>** section of an HTML page, within a **<style>** element

General form:

<style>

rule list

</style>

Example:

```
<html>
  <head>
    <style>
      body {background-color: powderblue;}
      h1   {color: blue;}
      p    {color: red;}
    </style>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```



◆ Levels of CSS

3. External:

- CSS rules are defined in a separate file (**.css file**)
- To use an external style sheet, add a link to it in the **<head>** section of the HTML page
- Example:

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

CSS file

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

HTML file

<style> @import url("styles.css"); </style>

◆ Levels of CSS

CSS conflict resolution:

When more than one style sheet applies to a specific tag in a document, the lowest level style sheet has precedence

1. Inline CSS
2. Internal CSS
3. External CSS
4. Browser default format

In the same level, the **last CSS rule** will be applied

◆ CSS Syntax

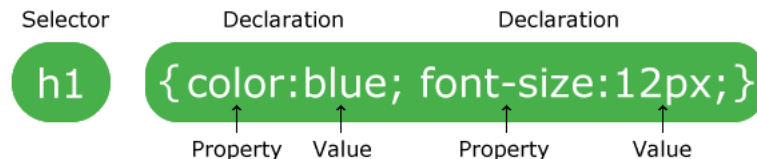
A **CSS block** (internal) or **CSS file** (external) consists of **CSS rules**

A **CSS rule** consists of a **selector** and a **declaration block** surrounded by a curly braces

The **selector** “selects” HTML elements in the webpage

A **declaration block** contains one or more **declarations**, separated by a semicolon

Each **declaration** includes a **CSS property name** and a **value**



Select and style

◆ CSS Syntax

CSS declaration: { property: value; ...}

CSS properties:

CSS3: more than 70 properties

CSS4: 129 properties

Categories (vary):

Fonts, lists, alignment, margins, colors, backgrounds, borders, positioning, etc.

Animations, backgrounds, box model, flexbox, grid, positioning, transitions, typography, etc.



◆ CSS Syntax

CSS property values: vary in forms

Keywords: left, right, small, large, etc.

Number (*with no space*)

Unit: px (*pixel*), in (*inches*), cm, mm, pt (*points*), pc (*picas, 12 pts*), em (*height of the letter 'm'*), ex (*height of the letter 'x'*), % (*percentage*), vw (*view port width*)

Color:

Name (140 color names)

Hex values, RGB (*red, green, blue*), RGBA (*A: alpha channel*), HLS (*hue, saturation, lightness*), HLSA

Special values: normal (*default value of the property*), inherit (*from parent element*), auto (*browser default value*)

Content

1. Introduction to CSS
2. CSS levels and syntax
- 3. CSS selectors**
4. CSS properties
5. Inheritance
6. The layout
7. The Box model
8. CSS convention

◆ Selectors

1. Simple selectors

Select HTML elements by name, class, or ID.

2. Attribute selectors

Select HTML elements based on their attributes or attribute values.

3. Combinator selectors

Select HTML elements based on a specific relationship between them.

4. Pseudo-class selectors

Select HTML elements based on a given state.

◆ HTML-Element Selector

Selects **all HTML elements** based on **element name** (HTML tag)

Syntax: **HTML-tag** { **property**: value; ... }

Example:

```
p {  
    text-align: center;  
    color: red;  
}
```

```
<p>Every paragraph will be  
affected by the style.</p>  
<p id="para1">Me too!</p>  
<p>And me!</p>
```

◆ Class Selector

Selects elements with a specific **class** attribute

Syntax: `.classname` { **property**: value; ... }

Example:

```
.center {
    text-align: center;
    color: red;
}
```

```
<h1 class="center">Red and
center-aligned heading</h1>
<p class="center">Red and
center-aligned paragraph.</p>
```

Combination of element and class selector:

Syntax: `HTML-tag.class-selector` (no space)

Selection: all HTML-tag elements that belong to the class specified by the class selector

```
p.center {
    text-align: center;
    color: red;
}
```

◆ ID Selector

Selects **one unique element** with a specific `id` attribute

Syntax: `#id` { **property**: value; ... }

Example:

```
#para1 {  
    font-weight: bold;  
}
```

```
<p id="para1">Hello World!</p>  
<p>This paragraph is not  
affected by the style.</p>
```

◆ Attribute Selector

Selects elements that **have specific attributes** or **attribute values**

Syntax:

- `[attribute]`: selects elements with specified attribute

```
[target] {
  background-color: yellow;
}
```

- `[attribute="value"]`: selects elements with a specified attribute and value

```
[target="_blank"] {
  background-color: yellow;
}
```

```
<a href="https://www.w3schools.com"
target="_self">W3schools.com</a>
<a href="https://www.w3schools.com/css/">CSS</a>
<a href="https://www.w3schools.com/html/"
target="_blank">HTML</a>
```

◆ Attribute Selector

Syntax (*cont.*):

- `[attribute~="word"]`: selects elements with an attribute value **containing** a specified **word** (*space-separated*)

```
[title~="flower"] {
  border: 5px solid yellow;
}
```

```



```

- `[attribute*="value"]`: select elements whose attribute value **contains** a specified value

```
[class*="te"] {
  background: yellow;
}
```

```
<div class="first_test">The first div element.</div>
<div class="second">The second div element.</div>
<div class="my-test">The third div element.</div>
<p class="mytest">This is some text in a paragraph.</p>
```

◆ Attribute Selector

Syntax (*cont.*):

- `[attribute="word"]`: selects elements with the specified attribute **starting** with the specified **word** followed by a hyphen (-)

```
[class="top"] {
  background: yellow;
}
```

```
<h1 class="top">Welcome</h1>
<p class="top-text">Hello world!</p>
<p class="topcontent">Are you learning CSS?</p>
```

- `[attribute^="value"]`: selects elements whose attribute **begins** with a specified **value**

```
[class^="top"] {
  background: yellow;
}
```

◆ Attribute Selector

Syntax (*cont.*):

- `[attribute$="value"]`: selects elements whose attribute value **ends** with a specified value

```
[class$="test"] {
  background: yellow;
}
<div class="first_test">The first div element.</div>
<div class="second">The second div element.</div>
<div class="my-test">The third div element.</div>
<p class="mytest">This is some text in a paragraph.</p>
```

Note: this selector can be **combine** with the element selector

Syntax: HTML-tag[attribute-selector]

Firstname:

Peter

```
<style>
input[type="text"] {
  width: 150px;
  padding: 6px;
  margin-bottom: 10px;
  background-color: pink;
}
</style>
```


◆ Special Selectors

Pseudo-classes:

Selects special elements that are not in the document tree

Prefixed with “:”

Pseudo	Selection	Example
:link	all unvisited links	a:link
:visited	all visited links	a:visited
:hover	element with the mouse pointer hovering	a:hover (hovered link)
:focus	element which has the focus	input:focus (focusing input)
:active	the active element	a:active (active link)
:first-child	the first child of its parent	p:first-child (first child <p>)

Some others: https://www.w3schools.com/cssref/css_selectors.asp

◆ Special Selectors

Pseudo-classes: Examples:

Link

```
<style>
/* unvisited link */
a:link {
  color: red;
}
/* visited link */
a:visited {
  color: green;
}
/* mouse over link */
a:hover {
  color: hotpink;
}
/* selected link */
a:active {
  color: blue;
}
</style>
```

Pseudo-classes can easily be combined with HTML classes

```
<style>
a.highlight:hover {
  color: #ff0000;
  font-size: 22px;
}
</style>
```

```
<input class="formField" type="text" />
```

```
input.formField{color:gray;}
input.formField:hover{color:green;}
input.formField:active{color:red;}
input.formField:focus{color:black;}
```

◆ Special Selectors

Universal selector:

Selects all elements in the webpage

Syntax: *

Example: apply red color to all elements in the webpage

```
* {  
  color: red;  
}
```

Example: CSS reset

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
  font-family: Arial, sans-serif;  
}
```

◆ Combinator Selectors

A CSS selector can contain more than one simple selector

Four CSS selector combinators:

1. Descendant selector (space)
2. Child selector ($>$)
3. Next sibling selector ($+$)
4. Subsequent-sibling selector ($\sim$)

Example:

```
div#header p em.required {  
  color: white;  
}
```



◆ Combinator Selectors

A CSS selector can contain more than one simple selector

Four CSS selector combinators:

1. Descendant selector (space)
2. Child selector ($>$)
3. Next sibling selector ($+$)
4. Subsequent-sibling selector ($\sim$)

Example:

```
div#header p em.required {  
  color: white;  
}
```



◆ Combinator Selectors

Descendant: select elements that are **descendants** of (**inside**) a specified element (*ascendant element*)

Syntax: ascendant-selector descendant-selector

Example:

```
div p {
  background-color: yellow;
}
```

white space character: space, tab,
line feed, form feed, new line

Child: selects all elements that are direct children of a specified element

Example:

```
div > p {
  background-color: yellow;
}
```

```
<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section>
    <!-- not Child but Descendant -->
    <p>Paragraph 3 in the div.</p>
  </section>
  <p>Paragraph 4 in the div.</p>
</div>
```

◆ Combinator Selectors

Next sibling: select an element that is directly after a specific element

Example:

```
div + p {
    background-color: yellow;
}
```

Subsequent-sibling: selects all elements that are next siblings of a specified element

Example:

```
div ~ p {
    background-color: yellow;
}
```

```
<div>
  <p>Paragraph 1 in the div.</p>
</div>
<p>Paragraph 3. After a div.</p>
<p>Paragraph 4. After a div.</p>
<div>
  <p>Paragraph 5 in the div.</p>
</div>
<p>Paragraph 7. After a div.</p>
<p>Paragraph 8. After a div.</p>
```

◆ Group Selectors

Group selectors:

To share the same declaration to multiple selectors (to save space)

Syntax: use comma (,) to separate the grouped selector

Example:

```
<style>
  a:hover, a:focus {
    background-color: green;
  }
  a:hover { color: red; }
  a:focus { color: yellow; }
</style>
```

```
h1, h2, p {
  text-align: center;
  color: red;
}
```


◆ Cascade Rule

Used to **solve the conflicts** occurring when there are two or more values for the same property on the same element

Sources of conflict:

- Conflicting values between **levels of style sheets**
- Within one style sheet (overlapped selectors)
- Inheritance can cause conflicts
- Property values can come from style sheets written by the document author, the browser user, and the browser defaults

◆ Cascade Rule

Cascade (specificity precedence) rules: **specificity**, **inheritance**, and **source order**

1. **ids** are more specific than **classes**; **classes** are more specific than **element names**
2. Style rules that **directly** target elements are more specific than style rules that are **inherited**
3. If elements have the same specificity, the **later rule** wins

```
div strong { color: red; }
strong { color: blue; }

<div>
  <strong>What color am I?</strong>
</div>
```

```
strong { color: red; }
strong { color: blue; }

<div>
  <strong>What color am I?</strong>
</div>
```

Content

1. Introduction to CSS
2. CSS levels and syntax
3. CSS selectors
- 4. CSS properties**
5. Inheritance
6. The layout
7. The Box model
8. CSS convention

◆ CSS Properties

1. Values
2. Units
3. Font
4. Text
5. Background
6. Position
7. Table
8. Lists
9. Cursor

◆ 1. Values

The value of a CSS property can be: *keyword*, *variable*

Keyword:

- *auto*: The browser automatically calculates the value.
- *inherit*: The attribute will use the same value as the parent element.
- *none*: Remove previously assigned values.
- *normal*: Using the default value of an attribute
- *transparent*: Transparent, allowing one component to be seen through to the components beneath it.

Variable: Numerical Values and Units

Fixed units like px (pixels), Responsive units like % (percentage of parent), etc.

2. CSS Units

Unit	Description
%	percentage
in	inch
cm	centimeter
mm	millimeter
em	1em is equal to the current font size. 2em means 2 times the size of the current font. E.g., if an element is displayed with a font of 12 pt, then '2em' is 24 pt. The 'em' is a very useful unit in CSS, since it can adapt automatically to the font that the reader uses
ex	one ex is the x-height of a font (x-height is usually about half the font-size)
pt	point (1 pt is the same as 1/72 inch)
pc	pica (1 pc is the same as 12 points)
px	pixels (a dot on the computer screen)

◆ 3. Fonts

Property	Description
font	Setting all properties for font in one declaration (font-style, font-variant, font-weight, font-size/line-height, font-family, etc.)
font-family	A prioritized list of [generic] font family names for an element
font-size	Sets the size of a font (absolute or relative), default: 16px/1em
font-style	Sets the style (normal italic oblique) for a text
font-variant	Specifies whether a text should be displayed in small-cap (normal small-caps)
font-weight	Sets the thickness of a font (normal bold bolder lighter <u>number</u>)

Some other font properties: font-size-adjust (*Firefox only*), font-stretch

◆ 3. Fonts

Font Families:

font-family: specifies the font for an element : "Serif" hay "Times New Roman"

Types:

Generic family: used to refer to a group of font families that are very similar, such as : "Serif" and "Monospace"

Font family: identify a specific font family such as: "Times New Roman" and "Arial"

Generic family	Font family	Description
Serif	Times New Roman Georgia	Serif fonts have small lines at the ends on some characters
Sans-serif	Arial Verdana	"Sans" means without - these fonts do not have the lines at the ends of characters
Monospace	Courier New Lucida Console	All monospace characters have the same width

◆ 3. Fonts

Font Families:

- should hold several font names as a "fallback" system
- Always start with the font you want, and always end with a generic family
- **Note:** If the font name is more than one word, it must be in quotation marks, like: "Times New Roman".

Example:

```
.p1 {  
    font-family: "Times New Roman", Times, serif;  
}
```

◆ 3. Fonts

CSS3 @font-face: This allows web browsers to use fonts not pre-installed on the user's computer by directly downloading the necessary fonts from the web server.

Syntax:

```
@font-face{  
    font-family: <my-font-name>;  
    src: url(<url>);  
}
```

Example:

```
@font-face{  
font-family: myFirstFont;  
src: url(bybsy-regular-webfont.ttf);}  
p{font-family:myFirstFont;}
```

◆ 3. Fonts

Font Style:

font-style: specifies the font style for a text

VALUES : **normal** | **italic** | **oblique** | **inherit**

italic and **oblique** are quite similar, but **oblique** is less supported

Example:

font-style: italic;

Italic oblique

Font Size:

font-size: specify the size of the text/font

VALUE: **<length>** | **<percentage-parents-font-size>** | **smaller** | **larger** | **xx-small** | **x-small**
| **small** | **medium** | **large** | **x-large** | **xx-large** | **inherit**

Example:

font-size: 12px;

6px 12px 18px 24px

◆ 3. Fonts

font-weight: specifies how thick or thin characters in text should be displayed

VALUES: **normal** | **bold** | **bolder** | **lighter** | **100-900** | **inherit**

Example:

font-weight: bold;

Bold

font-variant: specifies whether or not a text should be displayed in a small-caps font

VALUES: **normal** | **small-caps** | **inherit**

Example:

font-variant: small-caps;

SMALL CAPS

◆ 4. Text

Property	Value
color	color-value (color name rgb rgba hsl hsla)
text-align	left right center justify
text-decoration	line (none underline overline line-through) color style (solid double dotted dashed wavy)
text-indent	length (fixed or relative)
text-justify	auto inter-word inter-character none
text-overflow	clip ellipsis <i>string</i> (to represent the clipped text)
text-shadow	h-shadow v-shadow [blur-radius] color (e.g. 2px 2px 1px black)
text-transform	none capitalize uppercase lowercase

Other properties: direction, line-height, letter-spacing, word spacing, etc.

◆ 4. Text

Text properties allow you to format a block of text, including: : *color*, *letter-spacing*, *word-spacing*, *line-height*, *white-space*, *text-align*, *vertical-align*, *text-indent*, *text-decoration*, *text-transform*

color :

VALUES: <color> | inherit *color: rgb(128,0,0);*

Example: text is maroon

letter-spacing : the space between two characters in text

VALUES: <length> | <percentage-font-size>% | inherit

Example:

letter-spacing: .1em;

letters are spaced apart

◆ 4. Text

word-spacing : specify the space between the words

VALUES: **normal** | **<length>** | **inherit**

Example:

word-spacing: 40px;

words are spaced apart

line-height : specify the space between lines

VALUES: **normal** | **<number>** | **<length>** | **<percentage-font-size>%** | **inherit**

Example:

line-height: 2;

One thing was certain, that
the white kitten had had
nothing to do with it: -- it
was the black kitten's fault
entirely.

1

One thing was certain, that
the white kitten had had
nothing to do with it: -- it
was the black kitten's fault
entirely.

1.5

One thing was certain, that
the white kitten had had
nothing to do with it: -- it
was the black kitten's fault
entirely.

2

◆ 4. Text

white-space : specifies how white-space inside an element is handled

VALUES: **normal** | **pre** | **nowrap** | **pre-wrap** | **pre-line**

Example:

white-space: pre;

multiple spaces are not collapsed

text-indent : specify the indentation of the first line in a text-block

VALUES: **<length>** | **<percentage-width>%** | **inherit**

Example:

text-indent: 1em;

One thing was
certain, that the white
kitten had had nothing to
do with it: – it was the black
kitten's fault entirely.

◆ 4. Text

vertical-align : sets the vertical alignment of an element

VALUES: **baseline** | **sub** | **super** | **top** | **text-top** | **middle** | **bottom** | **text-bottom** | **<percentage-line-height>%** | **<length>** | **inherit**

Example:

vertical-align: sub;

X^{super} X_{sub}

text-align : et the horizontal alignment of a text

VALUES: **left** | **center** | **right** | **justify** | **inherit**

Example:

text-align: center;

One thing was certain, that
the white kitten had had
nothing to do with it: -- it
was the black kitten's fault
entirely.

Left

One thing was certain, that
the white kitten had had
nothing to do with it: -- it
was the black kitten's fault
entirely.

Center

One thing was certain, that
the white kitten had had
nothing to do with it: -- it
was the black kitten's fault
entirely.

Right

One thing was certain, that
the white kitten had had
nothing to do with it: -- it
was the black kitten's fault
entirely.

Justified

◆ 4. Text

text-decoration: control the appearance(**line-through** , **underline** , **overline**)

VALUES: **none** | **line-through** | **underline** | **overline**

Example:

text-decoration: strike-through;

One thing was certain, that
the white kitten had had
nothing to do with it — it
was the black kitten's fault
entirely.

Line-through

One thing was certain, that
the white kitten had had
nothing to do with it — it
was the black kitten's fault
entirely.

Underline

One thing was certain, that
the white kitten had had
nothing to do with it — it
was the black kitten's fault
entirely.

Overline

text-transform : control the capitalization of text (**uppercase**), (**lowercase**),
(**capitalize**)

VALUES: **lowercase** | **capitalize** | **uppercase** | **none** | **inherit**

Example:

text-transform: uppercase;

text case

Lowercase

Text Case

Capitalize

TEXT CASE

Uppercase

◆ 5. Background

Property	Value
background	bg-color bg-image position/bg-size bg-repeat bg-origin bg-clip bg-attachment (e.g. background:url(smiley.gif) 10px 20px/50px 50px;)
background-color	color-value (color name rgb rgba hsl hsla)
background-image	image-url
background-position	position-name (e.g. left top left center etc.) x% y% (relative) xpos ypos (absolute)
background-size	auto (origin size) <u>length</u> (width height) cover contain
background-repeat	repeat repeat-x repeat-y no-repeat
background-origin	padding-box border-box content-box
background-clip	Painting area: border-box padding-box content-box
background-attachment	scroll (with the page), fixed, local (scroll with the element content)

◆ 5. Background

background-color: property specifies the background color

VALUES: `<color>` | `transparent`

background-image: specifies an image to use as the background (png, jpeg, gif)

VALUES: `url("url")` | `none`

background-attachment: specifies whether the background image should scroll or be fixed

VALUES: `scroll` | `fixed`

```
div {  
    background-color: green;  
    opacity: 0.3;  
}
```

```
div {  
    background: rgba(0, 128, 0, 0.3)  
}
```

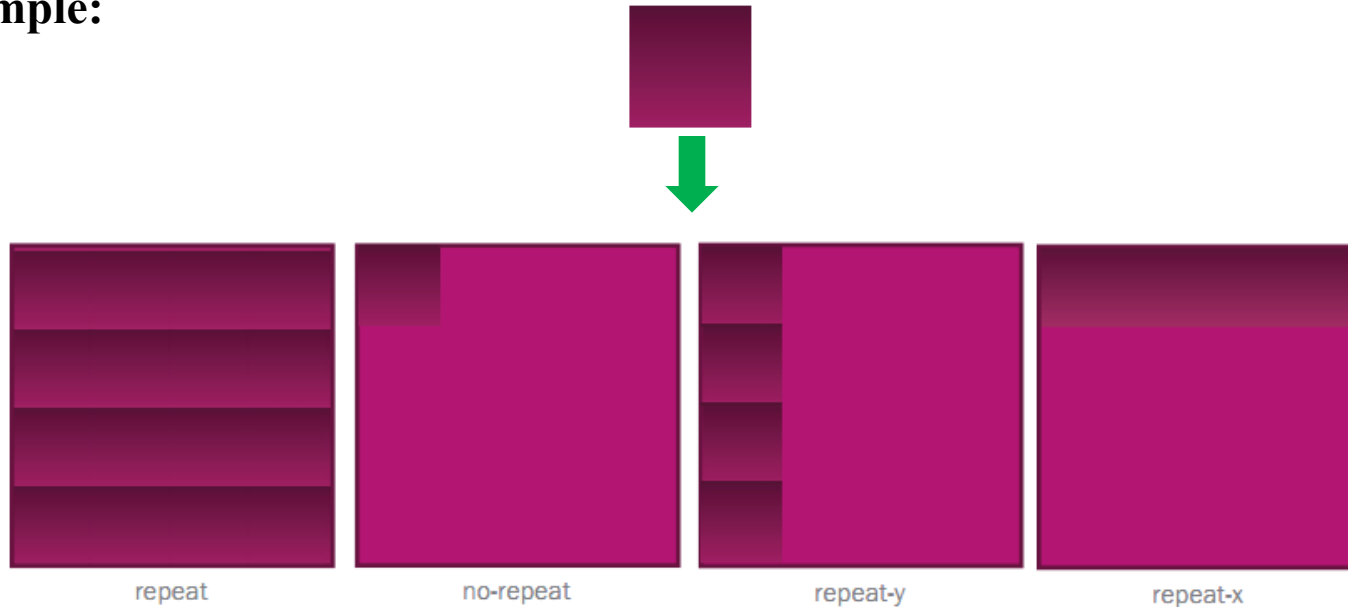
```
body {  
    background-image: url("img_tree.png");  
    background-repeat: no-repeat;  
    background-position: right top;  
    background-attachment: fixed;  
}
```

◆ 5. Background

background-repeat: sets if/how a background image will be repeated. By default, a background-image is repeated both vertically and horizontally

VALUES: **repeat** | **repeat-x** | **repeat-y** | **no-repeat**

Example:



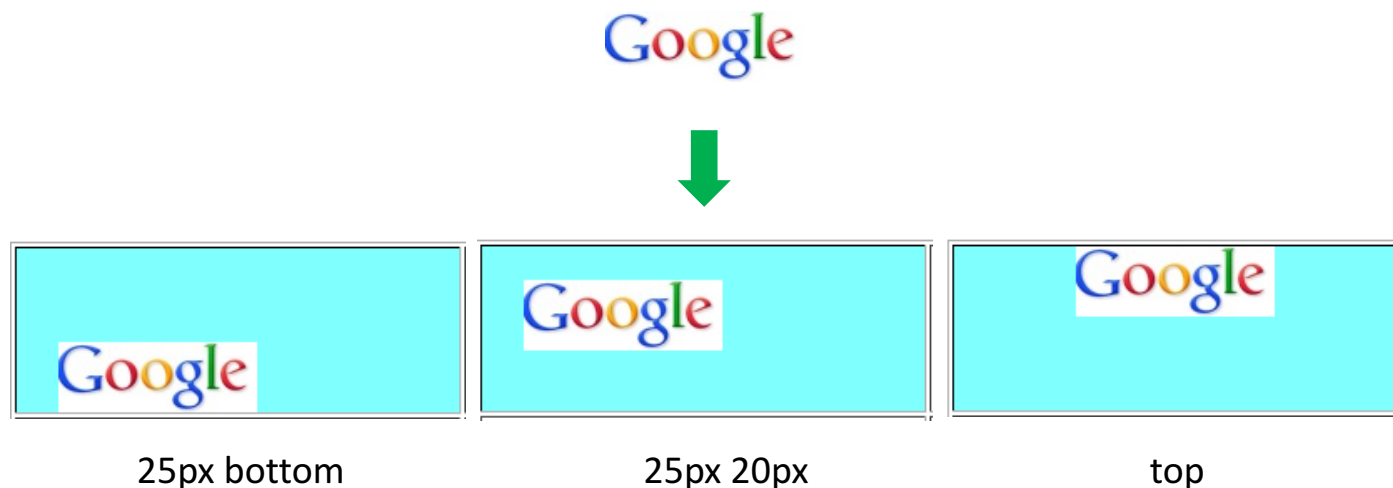
◆ 5. Background

background-position: set the starting position of the background image.

By default, a background-image is placed at the top-left corner

VALUES: `<length>` | `<percentage-box-width+padding>%` | `<left | right | center>` `<top | bottom | center>`

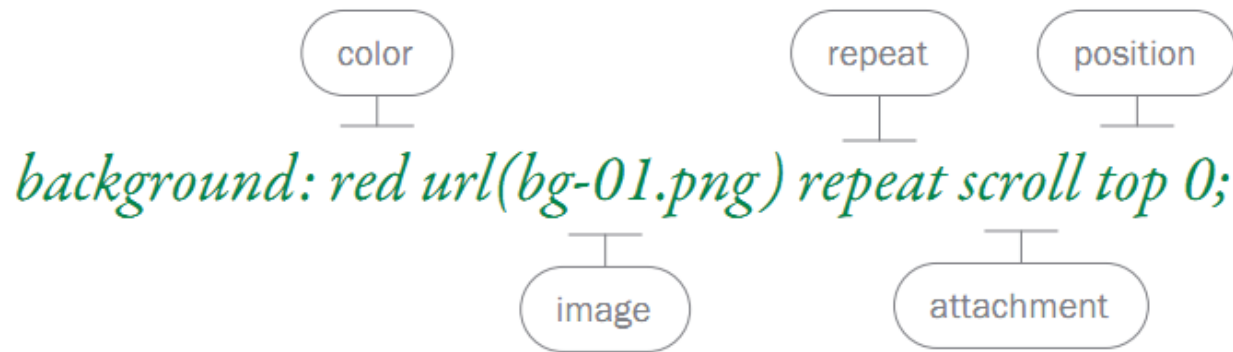
Example:



◆ 5. Background

background (shorthand property): specify all the background properties in one single property

VALUES: <background-color> <background-image> <background-repeat>
<background-attachment> <background-position> | none



◆ 6. Position

position: controlling the placement of elements

VALUES: **static** | **relative** | **absolute** | **fixed**

- **static**: positioned static by default (not affected by **top**, **bottom**, **left**, **right**)
- **relative**: positioned relative to its normal position, **top**, **bottom**, **left**, **right** properties will cause the element to be adjusted away from its normal position
- **absolute**: positioned relative to the nearest positioned ancestor
- **fixed**: positioned relative to the viewport, it always stays in the same place even if the page is scrolled

Note: An element with the position: **relative**, **absolute**, or **fixed** will have the ability to overlay another element.

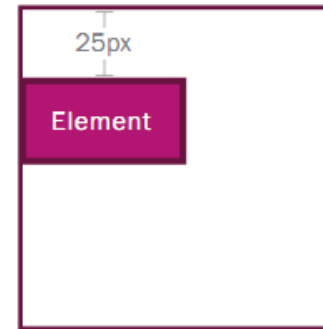
◆ 6. Position

Properties affect the position:

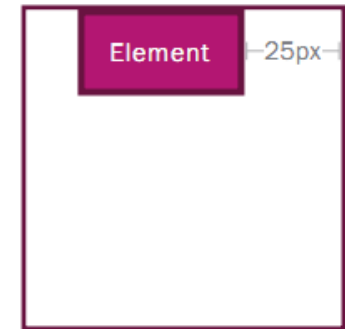
- **top**: Move the element a certain distance from the top edge
- **right**: Move the element a certain distance from the right edge
- **bottom**: Move the element a certain distance from the bottom edge
- **left**: Move the element a certain distance from the left edge

VALUES: **auto** | **<length>** |
<percentage-parents-width>%

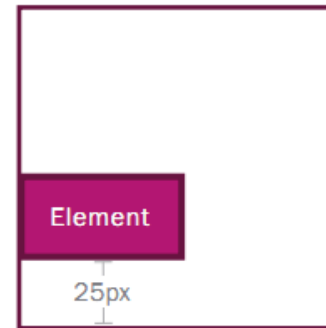
`<span id="e2">Element</span>`



top: 25px



right: 25px



bottom: 25px

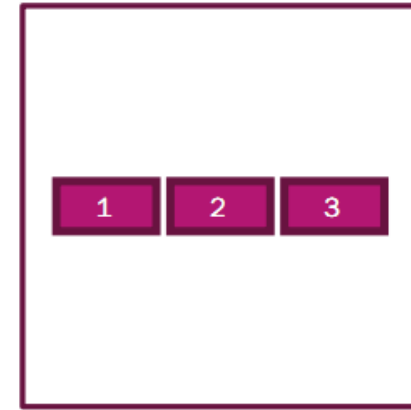


left: 25px

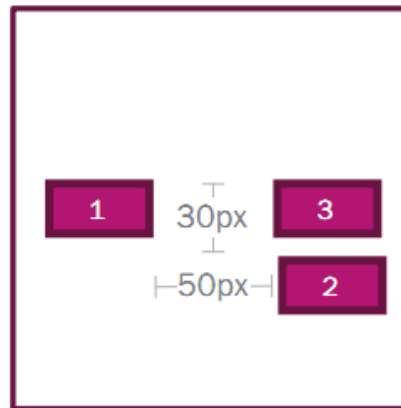
◆ 6. Position

Example: `<span id="e1">1</span>`
`<span id="e2">2</span>`
`<span id="e3">3</span>`

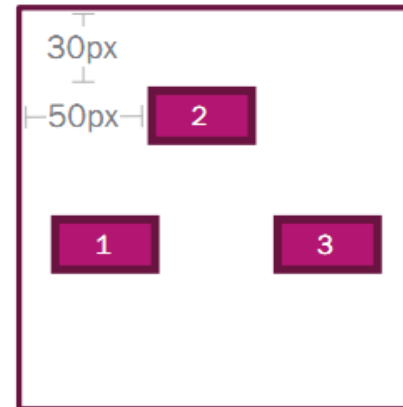
```
#e2 {  
  top:30px;  
  left:50px;  
}
```



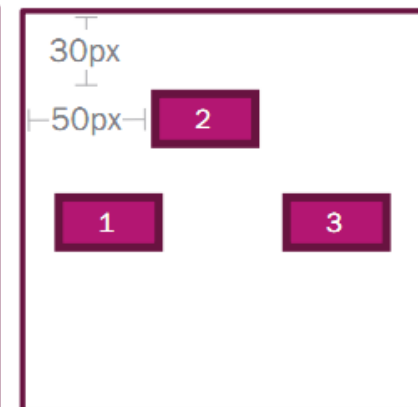
static



relative



absolute



fixed

7. Table

Property	Values
border	border-width border-style border-color
border-width	top right bottom left (name or value, e.g. thin medium thick 10px;)
border-style	top right bottom left (none hidden dotted dashed solid double groove ridge inset outset)
border-color	top right bottom left
border-collapse	separate collapse
border-spacing	<i>length</i> <i>h-length v-length</i>
caption-side	top bottom
empty-cell	show hide
table-layout	auto (cells' width belongs to the content) fixed (equal column width if the column width is not set; otherwise, uses the width of the 1 st row)

7. Table

border: specify the width, style, and color of table borders

```
table, th, td {
  border: 1px solid green;
}
```

Firstname	Lastname
Peter	Griffin
Lois	Griffin

border-collapse: sets whether table borders should collapse into a single border or be separated as in standard HTML

VALUES: **collapse** | **separate**

```
table {
  border-collapse: collapse;
}
```

Firstname	Lastname
Peter	Griffin
Lois	Griffin

7. Table

caption-side: caption position

VALUES: **top** | **bottom**

caption-side: top;

top

bottom

table-layout: layout of a table

VALUES: **auto** | **fixed** | **inherit**

- auto (cells' width belongs to the content)
- fixed (equal column width if the column width is not set; otherwise, uses the width of the 1st row)

7. Table

Border padding and border spacing

With Padding

hello	hello	hello
hello	hello	hello
hello	hello	hello

With Spacing

hello	hello	hello
hello	hello	hello
hello	hello	hello

◆ 8. List

Used to create styles for list elements(,)

list-style-type: specifies the type of list-item marker in a list

VALUES: **disc** | **circle** | **square** | **decimal** | **decimal-leading-zero** | **upper-roman** | **lower-roman** | **upper-alpha** | **lower-alpha** | **lower-greek** | **none** | **inherit**

Property	Values
list-style	list-style-type list-style-position list-style-image
list-style-type	none disc circle square decimal decimal-leading-zero lower-roman upper-roman lower-alpha upper-alpha <i>etc.</i>
list-style-position	inside outside
list-style-image	none url(...)
marker-offset	auto length

◆ 8. List

list-style-image: replace the list-item marker with an image

VALUES: `url( <url> )` | `none`

```
ul {
    list-style-image: url('sqpurple.gif');
    list-style-type: square;
}
```

list-style-position: specifies the position of the list-item markers

VALUES: `inside` | `outside`

list-style (shorthand property): used to set all the list properties in one declaration:

VALUES: `<list-style-type> <list-style-position> <list-style-image>`

```
ul {
    list-style: square inside url("sqpurple.gif");
}
```


◆ 8. List

list-style-type: square;

- | | | |
|----------|---------------|-------------------------|
| • Disc | A Upper-Alpha | 1 Decimal |
| ○ Circle | a Lower-Roman | 01 Decimal-Leading-Zero |
| ▪ Square | α Lower-Greek | I Upper-Roman |
| | | i Lower-Roman |

list-style-position: inside;

Outside:

- Coffee
- Tea
- Coca-cola

Inside:

- Coffee
- Tea
- Coca-cola

◆ 9. Cursor

cursor: Set the shape for the cursor

VALUES: **default** | **url(<url>)** | **auto** | **crosshair** | **pointer** | **move** | **e-resize** | **ne-resize** | **nw-resize** | **n-resize** | **se-resize** | **sw-resize** | **s-resize** | **w-resize** | **text** | **wait** | **help** | **progress** | **inherit**



crosshair



pointer



move



text



wait



help



progress



n-resize



e-resize



s-resize



w-resize



ne-resize



nw-resize



se-resize

Content

1. Introduction to CSS
2. CSS levels and syntax
3. CSS selectors
4. CSS properties
- 5. Inheritance**
6. The layout
7. The Box model
8. CSS convention

Inheritance

CSS styles are inherited from parent to child

Instead of selecting all elements individually

```
a, h1, p, strong {  
    font-family: Helvetica;  
}
```

You can style the parent and the children will inherit the styles

```
body {  
    font-family: Helvetica;  
}  
  
h2, h3 {  
    font-family: Consolas;  
}
```

◆ Exception in Inheritance

While many CSS styles are inherited from parent to child, not all CSS properties are inherited

```
<a href="/home">
  Back to <em>Home</em>
</a>
```

+

```
a {
  display: block;
  font-family: Arial;
}
```

➔

Back to Home

 inherits the **font-family** property, but not the **display**

There's **no rule** for what properties are inherited or not: the inheritance behavior defined in the **CSS specification**

Initial value	inline
Applies to	all elements
Inherited	no

<https://developer.mozilla.org/en-US/docs/Web/CSS/display>

◆ User Agent Styles

```
<h1>Chocolate</h1>
<p>
  <a href="...">CSS</a> is wonderful
</p>
```

+

```
body {
  color: red;
  font-family: Helvetica;
}
```



Chocolate

CSS is wonderful



Chocolate

CSS is wonderful



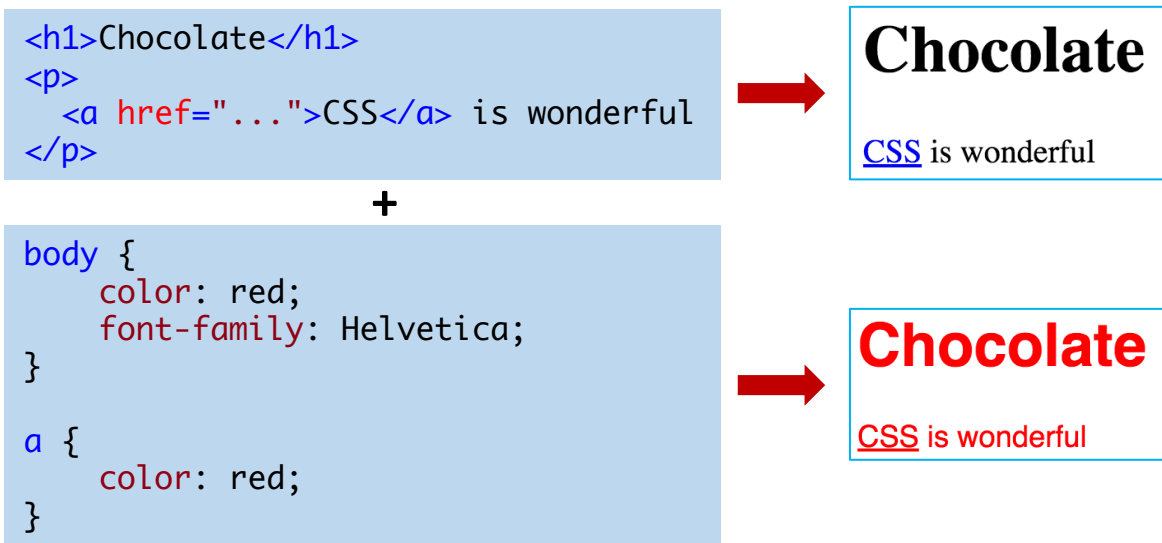
<a> inherits
font-family, but
not the color!

This is because the browser has its own default styles
Browser loads **its own default stylesheet** on every webpage

```
<html>
  <head>
    <title>User Agent Style</title>
    <link rel="stylesheet" href="user-agent-style.css"/>
    ...
  </head>
```

◆ User Agent Styles

So, to style the `<a>` links, we need to **override** the browser default style by **explicitly setting** a color:



Content

1. Introduction to CSS
2. CSS levels and syntax
3. CSS selectors
4. CSS properties
5. Inheritance
- 6. The layout**
7. The Box model
8. CSS convention

◆ Inline and Block Elements

The browser treats every element as a box

There are 2 types of boxes: **block** and **inline**

Block elements:

- Always start on a **new line**
- Fill up the horizontal from left to right of the page
- Example: `<p>`, `<h1>` to `<h6>`, `<div>`, `<ul>`, `<ol>`, `<pre>`, etc.

Inline elements:

- Doesn't start on a new line, i.e. one can **sit next to others**
- Doesn't respect to the top and bottom margins/paddings
- Example: `<img>`, `<a>`, `<input>`, `<span>`, etc.

◆ Inline and Block Elements

```
<html>
```

```
<body>
```

```
<p>Text and image 
```

```
and a button <input type="button" value="Click me!">
```

```
and a link to <a href="list.html">
```

```
an example</a>
```

```
and a
```

```
<h3>heading</h3>
```

```
and another text
```

```
and another
```

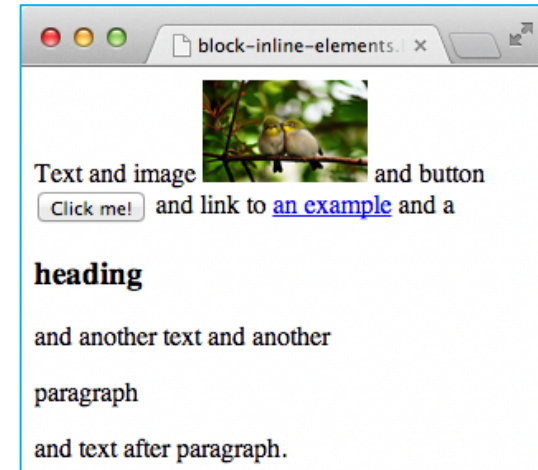
```
<p>paragraph</p>
```

```
and text after paragraph.
```

```
<p>
```

```
</body>
```

```
</html>
```



◆ Inline and Block Elements

To change the display mode of an element:

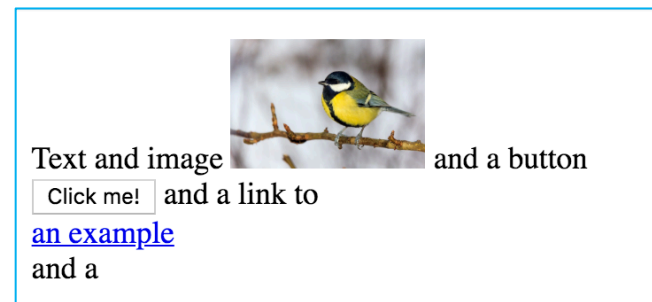
Use the ‘**display**’ property:

display: block | inline

Example:

```
a {  
    display: block;  
}
```

```
<p>Text and image   
and a button <input type="button" value="Click me!">  
and a link to <a href="list.html">an example</a>  
</p>
```

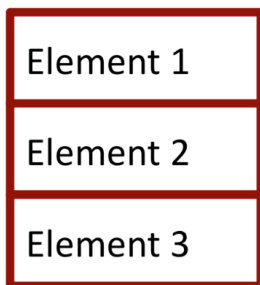


Display

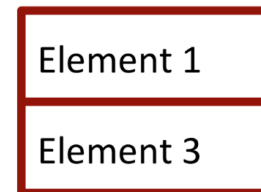
The ‘**display**’ property specifies how how element is displayed

Possible value: inline, block, list-item (same as block), etc.

```
<span id="s1">Element 1</span>
<span id="s2">Element 2</span>
<span id="s3">Element 3</span>
```



```
#s1, #s2, #s3 {
  display: block;
}
```



```
#s1, #s3 {display: block;}
#s2 {display: none;}
```



```
#s1, #s2, #s3 {display: inline;}
```

◆ Grouping Elements

Grouping elements lets us **style multiple elements** as a whole

Container elements are used to group elements:

****: **inline** container

```
<span>Hello <a href="...">World</a>!</span>
```

<div>: **block** container

```
<div>Hello <a href="...">World</a>!</div>
```

Width and **height** properties can be used to resize the block

◆ Overflow

The content of an element may **overflows** (bigger) its container

Possible values for these properties:

- **visible**: the overflow is not clipped and rendered outside the element's box
- **hidden**: the overflow is clipped, the rest content is hidden
- **scroll**: the overflow is clipped but a scrollbar is added
- **auto**: if the overflow is clipped, a scrollbar will be added

You can use the overflow property when you want to have better control of the layout. The overflow property specifies what happens if content overflows an element's box.

◆ Overflow

The **overflow**, **overflow-x**, **overflow-y** properties are used to specify what happens

```
<div style="height:10px; background-color:yellow;">Hello</div>
```

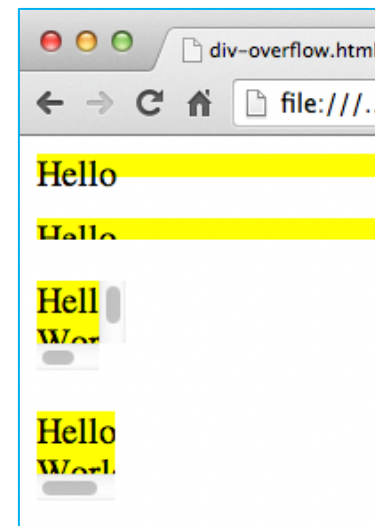
```
<br>
```

```
<div style="width:40px; height:10px;
background-color:yellow;
overflow:hidden;"> Hello</div>
```

```
<br>
```

```
<div style="width:40px; height:40px;
background-color:yellow;
overflow:auto;">Hello World</div> <br>
```

```
<div style="width:40px; height:40px;
background-color:yellow;
overflow-x:auto;
overflow-y:hidden;">Hello World</div>
```



◆ Visibility

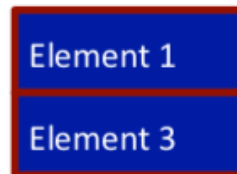
The ‘**visibility**’ property specifies whether an element is visible

VALUES: **visible** | **hidden** | **collapse** (used only with table row/column)

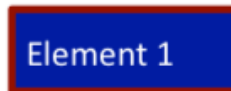
The ‘**opacity**’ property sets the opacity level for an element

VALUES: from **0.0** (fully transparent) to **1.0** (fully opaque)

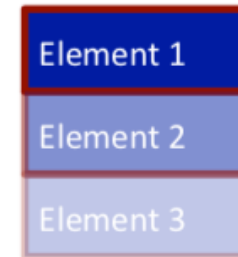
Note that, hidden elements (set with display property) still take up space on the page



```
#s1, #s3 {display: block;}  
#s2 {display: none;}
```



```
#s1, #s2, #s3 {display: block;}  
#s2 {visibility: hidden;}
```



```
#s1 {opacity: 1;}
```

```
#s1 {opacity: 0.5;}
```

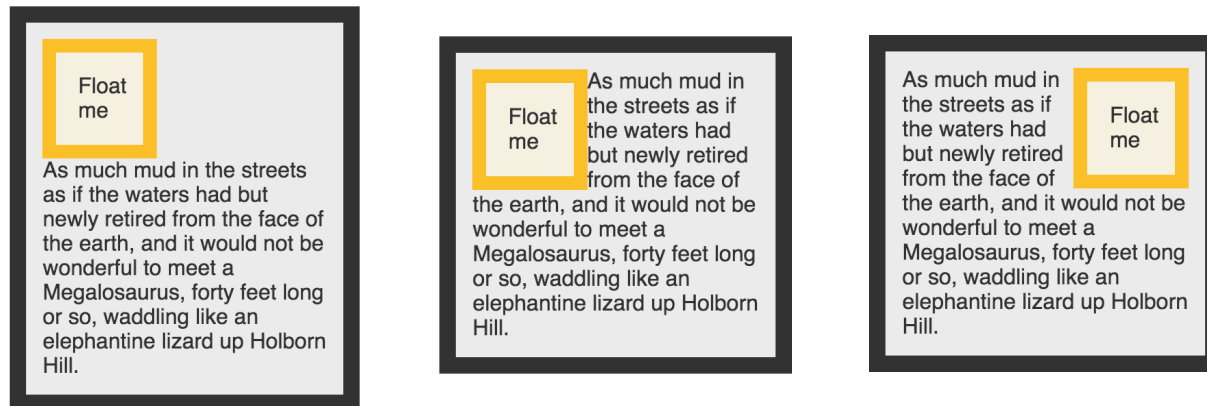
```
#s1 {opacity: 0.25;}
```


◆ Floating

The ‘**float**’ property is used to **float** boxes on the sides of other boxes and the siblings will **wrap around** the floating element

VALUES:

- **none** (default): the element doesn’t float
- **left**: the element floats to the left of its container
- **right**: the element floats to the right of its container



◆ Floating

The ‘**clear**’ property is used to specify that an element should **not wrap around** above floated elements

VALUES:

- **none** (wrap floating elements on both sides),
- **left** (doesn’t wrap the left element),
- **right** (doesn’t wrap the right element),
- **both** (doesn’t wrap both left and right)

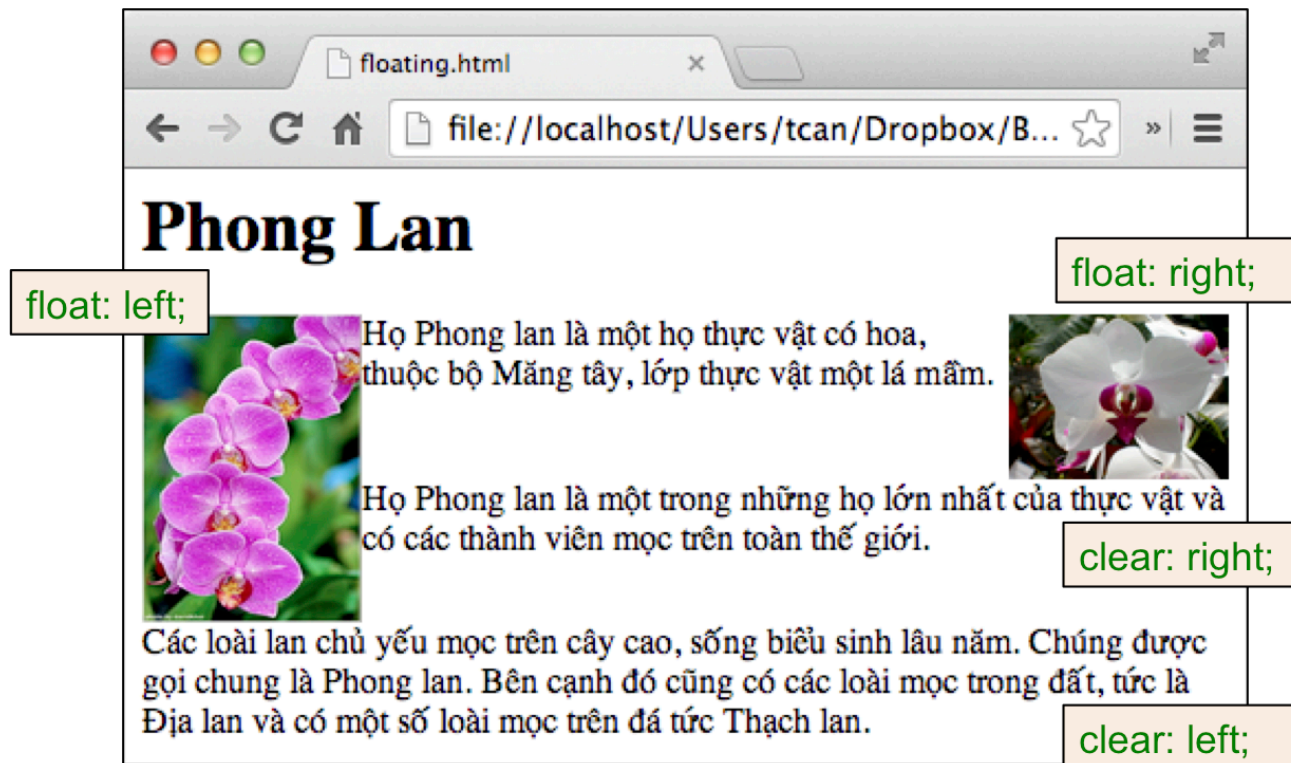
Float left block **This is the clear none block.** It lives alongside its floating siblings. It takes up the remaining space left in between. Float right

Float left **This is the clear left element.** It comes after the clear left, but remains alongside the float right. Tall float right block

Tall float left block **This is the clear right element.** It comes after the clear right, but remains alongside the float left. Float right

Float left **This is the clear both element.** It comes after both floating elements. Float right

♦ Floating



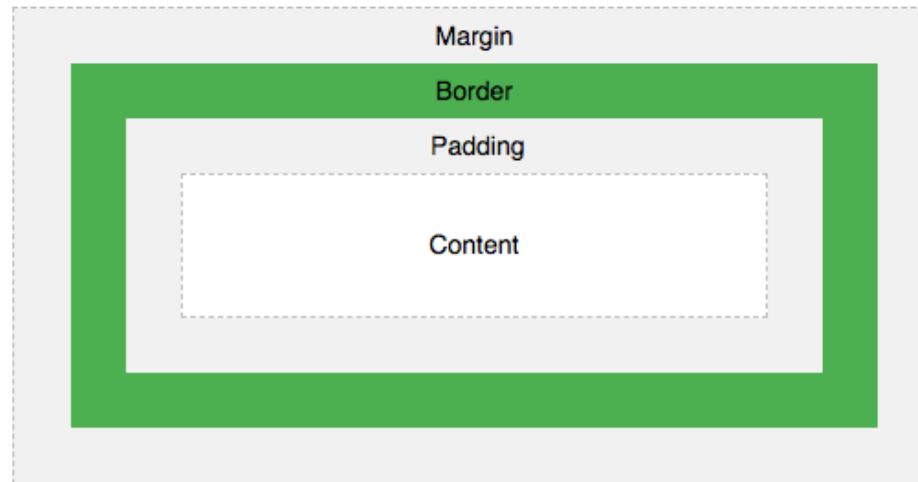
Content

1. Introduction to CSS
2. CSS levels and syntax
3. CSS selectors
4. CSS properties
5. Inheritance
6. The layout
- 7. The Box model**
8. CSS convention

◆ The Box Model

Each element can be considered as a **box** in the page layout

The box model is essentially a box that wraps around an element consisting of margins, borders, paddings, and content



◆ The Box Model – The Margin

Clears an area outside the border

The margin is transparent

The ‘margin’ property is used to set the size of the margin

margin: <top> [<right> [<bottom> [<left>]]]>

margin-top/right/bottom/left: <value> | auto

If the margin is **auto** and the box’s width is fixed, the margin will be set as large as possible

◆ The Box Model – The Border

A border goes around the padding and content

A border has a color, style, and the thickness

To set the border properties:

border: [thickness] <style> [color]

border-width: <value> | <top> [right] [bottom] [left]

border-style: none|hidden|dotted|dashed|solid|double
|groove|ridge|inset|outset

border-color: <color> | <top> [right] [bottom] [left]

◆ The Box Model – The Border

`width:340px; margin:auto; border-style: solid`

`width:340px; margin-left:auto; margin-right: 5px;`

`border-style:dashed; border-color:cyan yellow green blue;`

`border-style:groove;`

`border-style:outset;`

`border-style:inset;`

`border-style:ridge;`

◆ The Box Model – The Padding

Padding clears an area around the content

The padding is transparent (affected by the content background color)

To set the padding property:

`padding: size | <top> [right] [bottom] [left]`

`padding-top/right/bottom/left: value`

Demonstrating the Box Model



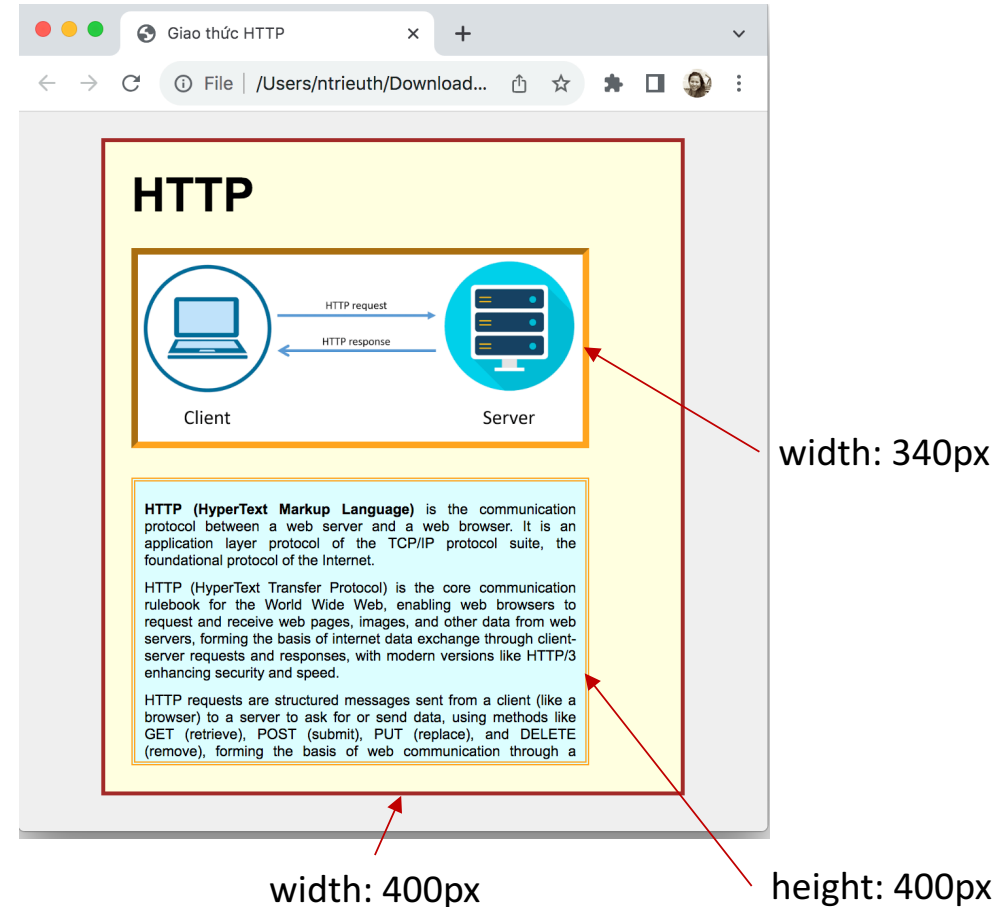
```
.box1 {  
  background-color: lightgrey;  
  width: 200px;  
  border: 20px solid green;  
  padding: 15px;  
  margin: 20px;  
}
```

```
.box2 {  
  background-color: lightgrey;  
  width: 200px;  
  border: 10px solid green;  
  padding: 5px;  
  margin: 10px;  
}
```

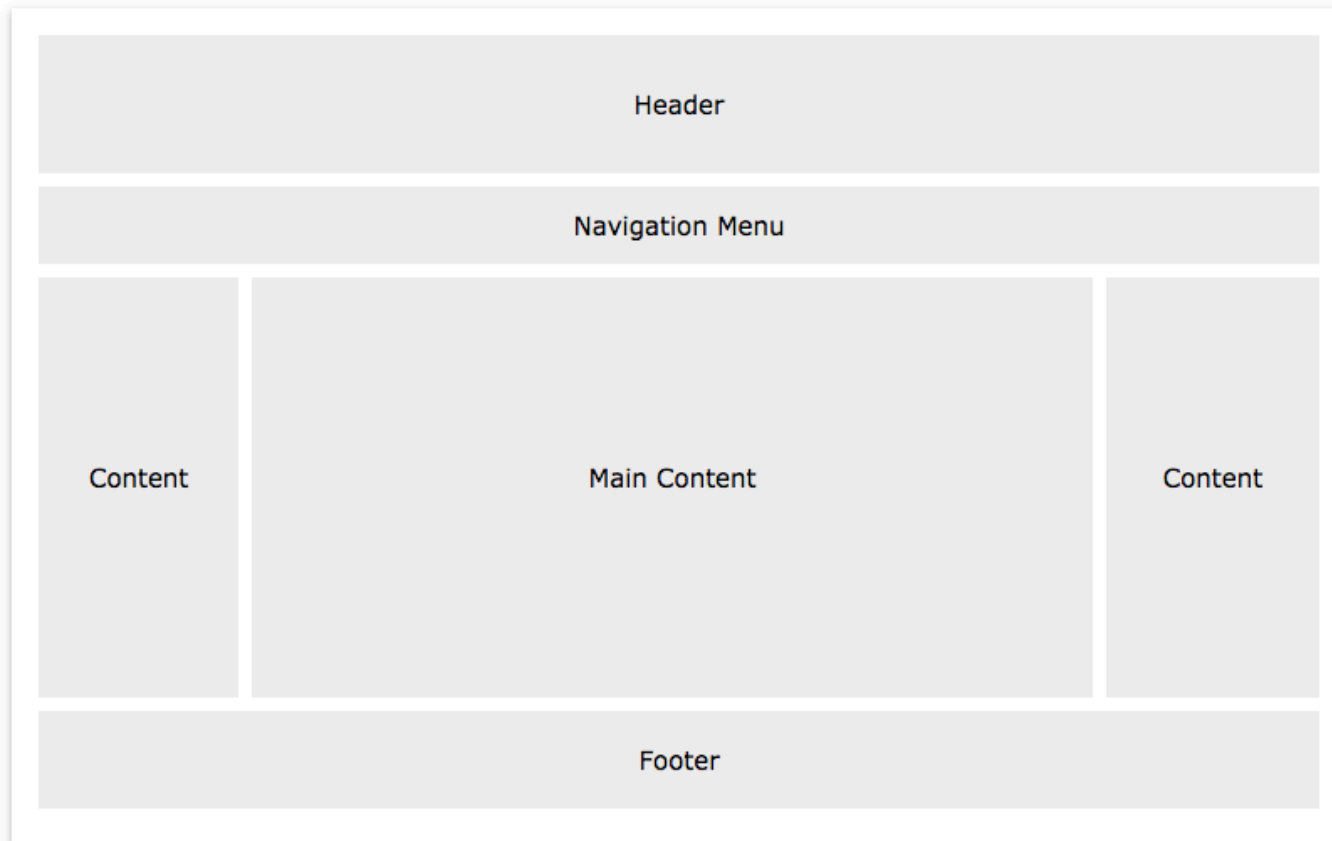
◆ The Box Model – Exercise

Design the following webpage:

1. Create the boxes and insert the content into these boxes. Ignore the format in this step.
2. Format the boxes:
 - **Outer box:** width 400px, background color LightYellow, border attribute 3px solid brown. You can add padding between the content and the border.
 - **Image border:** style inset, width 5px, and Orange color, margin-bottom as 20px.
 - **Inner box (containing text):** height 200px, border-style as double, and background as LightCyan. Note: use the overflow attribute appropriately to avoid the content flowing out of the box.



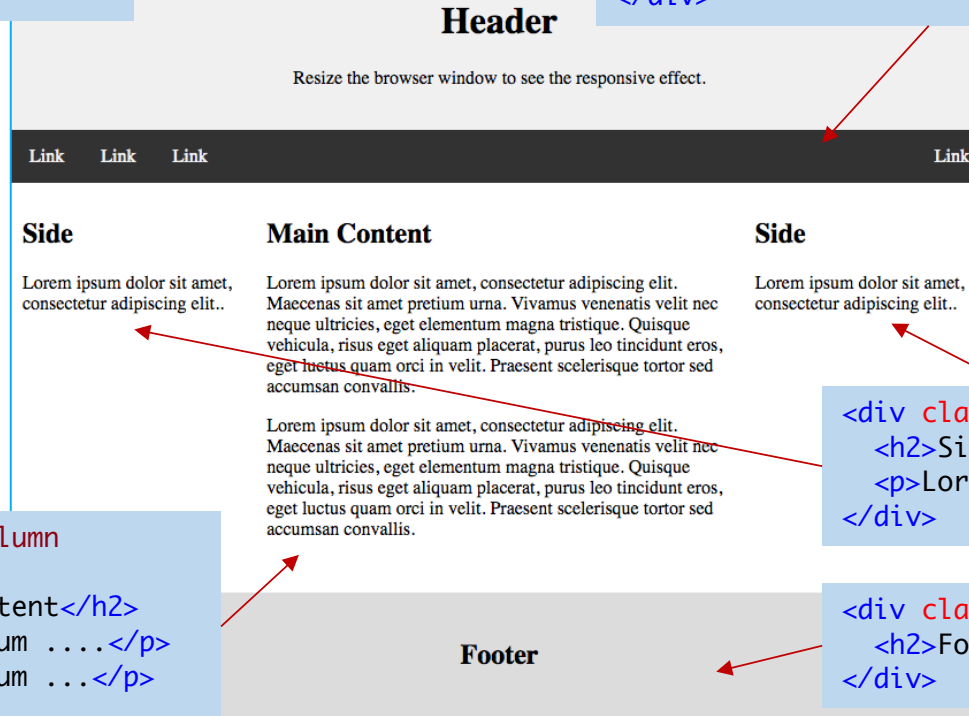
◆ A Typical Page Layout



A Typical Page Layout

```
<div class="header">
  <h1>Header</h1>
  <p>Resize...</p>
</div>
```

```
<div class="topnav">
  <a href="#">Link</a>
  <a href="#">Link</a>
  <a href="#">Link</a>
  <a href="#" class="navright">Link</a>
</div>
```



```
<div class="column
middle">
  <h2>Main Content</h2>
  <p>Lorem ipsum ....</p>
  <p>Lorem ipsum ...</p>
</div>
```

```
<div class="column side">
  <h2>Side</h2>
  <p>Lorem ipsum ...</p>
</div>
```

```
<div class="footer">
  <h2>Footer</h2>
</div>
```

◆ A Typical Page Layout

```
/* Style the header */
```

```
.header {  
    background-color: #f1f1f1;  
    padding: 20px;  
    text-align: center;  
}
```

```
/* Change color on hover */
```

```
.topnav a:hover {  
    background-color: #ddd;  
    color: black;  
}
```

```
/* Style the right top nav links */
```

```
a.rightnav{  
    float: right;  
}
```

```
/* Style the top navigation bar */
```

```
.topnav {  
    overflow: hidden;  
    background-color: #333;  
}
```

```
/* Style the topnav links */
```

```
.topnav a {  
    float: left;  
    display: block;  
    text-align: center;  
    padding: 14px 16px;  
    text-decoration: none;  
}
```

◆ A Typical Page Layout

```
/* Create three unequal columns */
```

```
.column {  
    float: left;  
    padding: 10px;  
}
```

```
/* Left and right column */
```

```
.column.side {  
    width: 25%;  
}
```

```
/* Middle column */
```

```
.column.middle {  
    width: 50%;  
}
```

```
/* Footer */
```

```
.footer {  
    padding: 20px;  
    text-align: center;  
    background: #ddd;  
    margin-top: 20px;  
}
```

Content

1. Introduction to CSS
2. CSS levels and syntax
3. CSS selectors
4. CSS properties
5. Inheritance
6. The layout
7. The Box model
- 8. CSS convention**

◆ Coding Convention

A CSS rule can be written as follow:

```
selector { property: value; ... }
```

However, it is recommended to use the following style:

```
selector {  
    property: value;  
    ...  
}
```

Using comments in CSS
is also recommended:

```
/* comment (multiple lines) */
```

```
/* Style the header */  
.header {  
    background-color: #f1f1f1;  
    padding: 20px;  
    text-align: center;  
}
```


◆ Naming Convention

Class name or ID should describe the semantic, not format

Example: use **warning** instead of **redbox**

Names are written in lowercase

Words are separated by a hyphen (-)

Example: **header-info** (instead of **headerInfo**)

Use prefix for subclass:

Example: **footer footer-logo, footer-copyright**

◆ Resources

CSS tutorial:

<https://www.w3schools.com/css/default.asp>

<https://www.tutorialspoint.com/css/>

<https://sass-lang.com/guide>: SCSS basics

CSS references:

<https://www.w3schools.com/cssref/default.asp>

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference>

CSS tools:

<https://html-css-js.com/css/generator/>: CSS code generator

<https://webcode.tools/css-generator/>



Question?

Chapter 3 – Cascading Style Sheet